Complete Deep Dive: Java Classes, Autoboxing, Abstract Classes & POJOs

---

1. Why Only One Public Class Per File?

The Problem Statement

Java enforces two strict rules:

1. Only **one public class** per `.java` file
2. That public class **must have the same name** as the file

To understand *why*, you need to understand how the JVM works.

The JVM's Job

The JVM (Java Virtual Machine) runs your code. But it doesn't magically know where to start. Every Java program has a single **entry point** — the `main` method:

```
public static void main(String[] args) { ... }
```

Everything in your program executes because of this one method. If you write 10,000 methods but never call them from `main`, they never execute. Period.

Why `main` Must Be `public`

The JVM lives *outside* your package. It needs to access your `main` method from outside. In Java, `public` means accessible from everywhere — even outside the package. So if `main` were `private` or `protected`, the JVM couldn't reach it. That's why `main` must be public.

Why `main` Must Be `static`

If `main` were a non-static method, the JVM would need to create an object of your class first before calling it. But to create an object, you need to run code — which creates a chicken-and-egg problem. `static` means the JVM can call it directly as `ClassName.main(...)` without creating any object.

The Confusion Problem (Why Only One Public Class)

Imagine Java *allowed* multiple public classes in one file:

```
// Demo.java — HYPOTHETICAL (not allowed)
public class Demo {
    public static void main(String[] args) { ... }
}

public class Sample {
    public static void main(String[] args) { ... }
}

public class SampleTwo {
    public static void main(String[] args) { ... }
}
```

Now the JVM looks at `Demo.java` and thinks: *"Which class do I load? Which `main` do I call? `Demo.main`? `Sample.main`? `SampleTwo.main`?"*

There's complete ambiguity. The JVM has no rule to resolve this.

By allowing only **one public class**, Java guarantees: the JVM looks at the filename, finds exactly one public class with that name, and calls its `main`. Zero ambiguity.

Why the Class Name Must Match the Filename

When the JVM needs to run `Demo.java`, it:

1. Looks at the filename: `Demo.java`
2. Compiles it to `Demo.class`
3. Loads `Demo` into memory
4. Calls `Demo.main(...)`

If the public class inside `Demo.java` were named `Sample`, the JVM would compile it to `Sample.class` but the file is called `Demo.java`. Now the JVM has a mismatch — it can't find `Demo.class` because it doesn't exist. Hence the rule: **class name = file name**.

What About 100-File Projects?

In real applications, you have hundreds of files. Each file has one public class. But only *one file* has the `main` method. You explicitly run *that* file:

```
javac Demo.java    # compile
java Demo          # run — JVM calls Demo.main()
```

The JVM doesn't search all 100 files for `main`. You tell it exactly which file to run. The other classes are imported and used *from within* that main class.

---

2. Wrapper Classes — Deep Dive

2. Wrapper Classes — Deep Dive

Java's Two Worlds

All data in Java belongs to one of two categories:

Primitives	Non-Primitives (Objects)
int	Integer
long	Long
short	Short
float	Float
double	Double
char	Character
boolean	Boolean
byte	Byte

What Are Wrapper Classes?

For every primitive, Java provides a corresponding **class**. These are called wrapper classes because they *wrap* the primitive value inside an object.

The conceptual internal structure of `Integer` looks roughly like this:

```
public final class Integer {

    private int value;  // the actual primitive stored inside

    // Constructor (deprecated in modern Java)
    public Integer(int value) {
        this.value = value;
    }
```

```
    // Static factory method (preferred)
    public static Integer valueOf(int x) {
        // uses caching internally (explained later)
        return new Integer(x);
    }

    // Unwrap: get the primitive back
    public int intValue() {
        return this.value;
    }

    // Compare values (not references)
    public boolean equals(Integer other) {
        return this.value == other.value;
    }
}
```

Memory Difference: Primitive vs Object

Primitive:

int x = 10;

Stack Memory

```
┌──────────────┐
│  x = 10      │
└──────────────┘
```

One container in stack. Direct. Fast.

Wrapper Object:

Integer x = new Integer(10);

```
Stack Memory          Heap Memory

┌──────────┐          ┌──────────────────┐
│ x = @1001 │ ──────► │ Integer object    │
└──────────┘          │   value = 10      │
                      │   metadata        │
                      │   padding         │
                      └──────────────────┘
```

Reference in stack, actual object in heap. More memory, more overhead.

Why Do Wrapper Classes Exist?

Reason 1 — Collections only work with objects:

Java's Collection Framework (ArrayList, LinkedList, HashMap, etc.) was designed to work with *objects only*, not primitives. Internally they use `Object` type. You cannot do this:

```
ArrayList<int> list = new ArrayList<>();  // COMPILE ERROR
```

You must do:

```
ArrayList<Integer> list = new ArrayList<>();  // Works!
```

So when you need to store numbers in a collection, you *need* the wrapper class.

Reason 2 — Extra functionality:

Wrapper classes come with useful methods that bare primitives don't have:

```
Integer.parseInt("42");        // String → int
Integer.toBinaryString(10);    // "1010"
Integer.MAX_VALUE;             // 2147483647
Integer.MIN_VALUE;             // -2147483648
Integer.compare(5, 10);        // comparison utility
```

Reason 3 — Null support:

Primitives cannot be null. Objects can. Sometimes you genuinely don't know a value — it's absent, not zero:

```
int age = null;      // COMPILE ERROR — primitives can't be null
Integer age = null;  // Works — null means "no value assigned"
```

Reason 4 — OOP philosophy:

Java is object-oriented. Having a class representation for every primitive keeps the language consistent and allows primitives to participate in OOP contexts.

Why Primitives Still Exist (Not Replaced by Wrappers)

Reason 1 — Legacy from C/C++:

Java was designed to attract C/C++ developers. Those languages have primitives. Keeping primitives made Java familiar and easier to adopt.

Reason 2 — Performance (the real reason):

```
int x = 10;           // Stack only. Direct value. Blazing fast.
Integer x = new Integer(10);  // Stack reference + Heap object + metadata
```

Objects carry overhead:

- Object header (~16 bytes in HotSpot JVM)
- Padding bytes for alignment
- Indirection (dereference the pointer to get value)
- Garbage collection overhead

For tight loops, mathematical computations, or high-performance code, primitives are significantly faster. The JVM can also optimize primitive code more aggressively.

---

3. Autoboxing — Complete Deep Dive

What Is Autoboxing?

Autoboxing is the **automatic conversion** of a primitive into its corresponding wrapper object, done by the Java compiler behind the scenes.

```
int x = 10;
```

```
Integer y = x;  // Autoboxing happens here
```

You wrote one simple line. But the compiler internally transforms it.

What the Compiler Actually Does

Old Java (pre-Java 5, deprecated approach):

```
Integer y = new Integer(x);
```

The compiler rewrote your code to use the constructor.

Modern Java (Java 5+, current approach):

```
Integer y = Integer.valueOf(x);
```

The compiler uses the static `valueOf` method instead. Why? Because `valueOf` uses caching (explained in detail later), making it more efficient.

You can verify: if you write `new Integer(10)` today, your IDE draws a strikethrough and warns: *"Constructor Integer(int) is deprecated since Java 9 and marked for removal."*

The `valueOf` Method

```
public static Integer valueOf(int x) {
    if (x >= -128 && x <= 127) {
        return IntegerCache.cache[x + 128];  // return cached object
    }
    return new Integer(x);  // create new object only if outside cache range
}
```

This is the magic. For values in the range -128 to 127, no new object is created — a pre-existing cached object is returned.

Autoboxing in Action

```
int x = 10;
Integer y = x;  // autoboxing

System.out.println(x);  // prints 10
System.out.println(y);  // prints 10 — unboxing happens here too!
```

When you print `y` (an `Integer` object), Java automatically calls `y.intValue()` to get the primitive and pass it to `println`. This is **unboxing**.

---

4. Unboxing — Complete Deep Dive

What Is Unboxing?

Unboxing is the **automatic conversion** of a wrapper object back into its primitive value.

```
Integer x = 20;     // autoboxing
int y = x;          // unboxing — compiler adds x.intValue() internally
```

What the Compiler Actually Does

Your code:

```
int y = x;
```

Compiler transforms it to:

```
int y = x.intValue();
```

The `intValue()` method is defined inside `Integer`:

```
public int intValue() {
    return this.value;  // returns the wrapped primitive
}
```

It's essentially a getter for the internal `value` field.

---

5. When Does Autoboxing/Unboxing Apply?

Case 1 — Assignment Statements

```
// Autoboxing (primitive → object)
int a = 5;
```

```
Integer b = a;  // compiler: Integer b = Integer.valueOf(a);

// Unboxing (object → primitive)
Integer c = 100;
int d = c;      // compiler: int d = c.intValue();
```

Case 2 — Method Calls (Arguments & Parameters)

```
static void printValue(int x) {
    System.out.println(x);
}

public static void main(String[] args) {
    Integer val = 50;
    printValue(val);  // unboxing: val.intValue() passed as int
}
```

The method expects `int`, you pass `Integer`. Java automatically unboxes it.

The reverse also works:

```
static void printValue(Integer x) {
    System.out.println(x);
}

public static void main(String[] args) {
    int val = 50;
    printValue(val);  // autoboxing: Integer.valueOf(val) passed as Integer
}
```

Case 3 — Arithmetic Operations

```
Integer a = 10;
Integer b = 20;
int sum = a + b;
```

The compiler transforms the addition:

```
int sum = a.intValue() + b.intValue();
```

You can't add two `Integer` objects directly in the mathematical sense. The JVM unboxes them first, adds the primitives, then assigns the result.

Case 4 — Comparison with Primitives

```
Integer a = 100;
int b = 100;
System.out.println(a == b);  // true — a is unboxed to int, then compared
```

When comparing an `Integer` with an `int` using `==`, the `Integer` gets unboxed first, then a primitive comparison happens.

---

6. NullPointerException in Unboxing

The Hidden Danger

Objects can be `null`. Primitives cannot. When you unbox a `null` object, you get a `NullPointerException` at **runtime** — not at compile time. The compiler won't warn you.

```
Integer x = null;   // valid — object can be null
int y = x;          // COMPILES FINE

System.out.println(y);  // NullPointerException at runtime!
```

Why It Crashes

The compiler transforms `int y = x` into `int y = x.intValue()`. But `x` is `null`. Calling any method on `null` throws `NullPointerException`. It's equivalent to:

```
int y = null.intValue();  // NPE — you're calling a method on null
```

The Lesson

Always check for null before unboxing:

```
Integer x = null;
if (x != null) {
    int y = x;
    System.out.println(y);
```

```
} else {
    System.out.println("Value is null");
}
```

This is especially important when:

- Getting values from a database (could be null)
- Working with `Map.get()` (returns null if key absent)
- Method return values that could be null

---

7. The `==` Operator vs `equals()` — Very Deep Dive

How `==` Works on Primitives

```
int x = 10000;
int y = 10000;
System.out.println(x == y);  // true
```

Memory layout:

Stack

```
┌──────────────────┐
│  x = 10000       │
│  y = 10000       │
└──────────────────┘
```

`x` and `y` are containers. `x == y` directly compares their contents: `10000 == 10000` → `true`. Simple.

How `==` Works on Objects

```
Integer a = 200;
Integer b = 200;
System.out.println(a == b);  // false !!
```

Memory layout:

```
Stack                    Heap
┌──────────────┐         ┌──────────────────┐
| a = @address1| ──────► | Integer(200)     |  @address1
| b = @address2| ──────► | Integer(200)     |  @address2
└──────────────┘         └──────────────────┘
```

`a` and `b` are **reference variables**. They store memory addresses, not values. `a == b` asks: *"Do `a` and `b` point to the same memory address?"* They don't — two separate objects were created. So `false`.

The `==` operator always compares whatever is directly stored in the variable:

- For primitives: the value itself
- For objects: the reference (memory address)

It never "looks inside" the object. That's not a special behavior — it's consistent behavior.

How to Actually Compare Object Values

Method 1 — Using `intValue()`:

```
Integer a = 200;
Integer b = 200;
System.out.println(a.intValue() == b.intValue());  // true
```

You manually unbox both and compare primitives.

Method 2 — Using `equals()`:

```
Integer a = 200;
Integer b = 200;
System.out.println(a.equals(b));  // true
```

The `equals()` method is designed to compare the *contents* of objects, not their references. Internally:

```
public boolean equals(Object obj) {
    if (obj instanceof Integer) {
        return this.value == ((Integer)obj).value;
```

```
    }
    return false;
}
```

It extracts the primitive `value` from both objects and compares those.

The Confusing Case: Why Does `==` Return `true` for Small Values?

```
Integer a = 100;
Integer b = 100;
System.out.println(a == b);  // true  ← surprising!

Integer c = 200;
Integer d = 200;
System.out.println(c == d);  // false ← expected
```

This is because of **Integer caching** (explained next). For values -128 to 127, the JVM reuses the same object. So `a` and `b` both point to the *same* cached object → same reference → `==` returns `true`.

---

8. Caching Inside Integer Class — Very Deep Dive

The Problem Caching Solves

Every time you write `Integer x = 5`, the compiler calls `Integer.valueOf(5)`. If this always created a new object:

```
for (int i = 0; i < 1000000; i++) {
    Integer x = 5;  // creates 1,000,000 objects all holding value 5!
}
```

Massive memory waste for commonly used small integers.

The Solution: IntegerCache

Inside the `Integer` class, there's a private static inner class called `IntegerCache`:

```
private static class IntegerCache {
    static final int low = -128;
```

```
    static final int high = 127;
    static final Integer[] cache;

    static {
        // Pre-create all Integer objects from -128 to 127
        cache = new Integer[(high - low) + 1];
        int j = low;
        for (int k = 0; k < cache.length; k++) {
            cache[k] = new Integer(j++);  // creates Integer(-128), Integer(-127)... Integer(127)
        }
    }
}
```

When the `Integer` class is first loaded by the JVM, this static block runs and creates 256 `Integer` objects (for -128 to 127) and stores them in an array.

How `valueOf` Uses the Cache

```
public static Integer valueOf(int i) {
    if (i >= IntegerCache.low && i <= IntegerCache.high) {
        return IntegerCache.cache[i + (-IntegerCache.low)];
        // For i=5: returns cache[5 + 128] = cache[133] = pre-created Integer(5)
    }
    return new Integer(i);  // only create new object if outside range
}
```

Memory Visualization

Heap Memory (on JVM startup, when Integer class loads)

```
Heap Memory (on JVM startup, when Integer class loads)
┌──────────────────────────────────────────────────────────┐
│  IntegerCache.cache[]                                    │
│  [0]  = Integer(-128)  ← @addr_0                         │
│  [1]  = Integer(-127)  ← @addr_1                         │
│  ...                                                     │
│  [128] = Integer(0)    ← @addr_128                       │
│  ...                                                     │
│  [228] = Integer(100)  ← @addr_228                       │
│  ...                                                     │
│  [255] = Integer(127)  ← @addr_255                       │
└──────────────────────────────────────────────────────────┘
```

Now when you write:

Integer x = 100; // Integer.valueOf(100) → returns cache[228] → @addr_228
Integer y = 100; // Integer.valueOf(100) → returns cache[228] → @addr_228

Both `x` and `y` point to the **same object** `@addr_228`.

Stack

```
┌──────────────────────────┐
│ x = @addr_228 ──────┼──────► Integer(100) in cache
│ y = @addr_228 ──────┼──────► same object!
└──────────────────────────┘
```

So `x == y` is `true` — not because their values match, but because they're literally the same object.

Why the Range -128 to 127?

This is the range of Java's `byte` data type, and these are statistically the most commonly used integer values in programs (loop counters, small constants, array indices, etc.). Caching them gives the most benefit for the least memory cost (only 256 objects pre-created).

The Rule to Remember

- For `-128 to 127`: `==` might return `true` (cache reuse), but this is **implementation-dependent**
- For outside this range: `==` will return `false` (new objects each time)
- **Always use `equals()` to compare Integer values** — never rely on `==` for correctness

---

9. Conceptual View of the Integer Class

Putting everything together, here's the complete conceptual model:

```
public final class Integer extends Number implements Comparable<Integer> {

    // 1. The actual value stored
    private final int value;

    // 2. Cache for -128 to 127
    private static class IntegerCache {
        static final Integer[] cache = new Integer[256];
        static {
            for (int i = 0; i < 256; i++) {
                cache[i] = new Integer(i - 128);
            }
        }
    }

    // 3. Deprecated constructor
    @Deprecated
    public Integer(int value) {
        this.value = value;
    }

    // 4. Preferred static factory (used by autoboxing)
    public static Integer valueOf(int i) {
        if (i >= -128 && i <= 127) {
            return IntegerCache.cache[i + 128];
        }
        return new Integer(i);
    }

    // 5. Unboxing method (used by unboxing)
    public int intValue() {
        return value;
```

```
    }

    // 6. Value comparison
    public boolean equals(Object obj) {
        if (obj instanceof Integer) {
            return value == ((Integer)obj).value;
        }
        return false;
    }

    // 7. Utility methods
    public static int parseInt(String s) { ... }
    public static String toBinaryString(int i) { ... }
    public static String toHexString(int i) { ... }
    public static int max(int a, int b) { ... }
    public static int min(int a, int b) { ... }

    // Constants
    public static final int MAX_VALUE = 0x7fffffff;  // 2147483647
    public static final int MIN_VALUE = 0x80000000;  // -2147483648
}
```

Key points:

- `final class` — cannot be subclassed (nobody should extend Integer)
- `final int value` — immutable once created
- `valueOf` is what autoboxing uses (not the constructor)
- `intValue` is what unboxing uses
- `equals` compares values, `==` compares references

---

10. Abstract Classes — Extremely Deep Dive

Recap: What Is an Abstract Class?

An abstract class is a class that **cannot be instantiated directly**. It exists to be **inherited**. It can contain:

- Abstract methods (declared but not implemented)
- Concrete methods (fully implemented)
- Fields/variables

- Constructors

```
abstract class Animal {
   String name;

   abstract void makeSound();  // no body — subclass must implement

   void sleep() {              // has body — subclass inherits this
      System.out.println("Sleeping...");
   }
}

class Dog extends Animal {
   @Override
   void makeSound() {
      System.out.println("Woof!");  // must implement abstract method
   }
}

Animal a = new Dog();   // OK — Dog is concrete
Animal a = new Animal(); // COMPILE ERROR — cannot instantiate abstract class
```

Interview Question 1: Can Abstract Classes Have Constructors?

Answer: YES

```
abstract class Animal {
   String name;

   // Constructor in abstract class — perfectly valid
   Animal(String name) {
      this.name = name;
   }

   abstract void makeSound();
}

class Dog extends Animal {
   Dog(String name) {
      super(name);  // calls Animal's constructor
   }

   @Override
   void makeSound() {
```

```
        System.out.println("Woof!");
    }
}

// Usage
Animal a = new Dog("Bruno");  // Dog constructor → calls Animal("Bruno")
```

Why it makes sense:

Even though you can't do `new Animal()`, when you do `new Dog("Bruno")`:

1. JVM creates a `Dog` object
2. `Dog`'s constructor is called
3. `Dog`'s constructor calls `super(name)` → `Animal`'s constructor runs
4. `Animal`'s constructor initializes `name = "Bruno"`

The constructor in the abstract class runs — just not through direct instantiation of the abstract class. It runs through the child's construction chain.

What if the abstract class has no explicit constructor?

Java provides a default no-arg constructor automatically:

```
abstract class Animal {
    // Java implicitly adds: Animal() { super(); }
}

class Dog extends Animal {
    // Java implicitly adds: Dog() { super(); }  ← calls Animal()
}
```

Interview Question 2: Can Abstract Classes Be `final`?

Answer: NO — Compile error

```
final abstract class Animal { }  // ERROR: illegal combination of modifiers
```

Why it's contradictory:

Keyword	Meaning

`abstract`	*"You MUST extend me"*
`final`	*"You CANNOT extend me"*

These two instructions directly contradict each other. Java refuses to compile this. An abstract class with no way to be extended is completely useless — its abstract methods can never be implemented, so no usable object can ever be created.

Interview Question 3: Can Abstract Classes Have Static Methods/Variables/Blocks?

Answer: YES — All three are allowed

```
abstract class Animal {

    // Static variable — belongs to the class, not any object
    static String kingdom = "Animalia";

    // Static block — runs when class is loaded
    static {
        System.out.println("Animal class loaded");
        kingdom = "Animalia";
    }

    // Static method — callable without any object
    static void classifyAnimal() {
        System.out.println("Kingdom: " + kingdom);
    }

    abstract void makeSound();
}
```

Why it works:

Static members belong to the **class**, not to any object instance. Since the `Animal` *class* exists (even though you can't create an `Animal` *object*), all static things work perfectly fine.

```
Animal.classifyAnimal();     // works — no object needed
Animal.kingdom = "Animalia"; // works — static field
```

One nuance: Static methods cannot be abstract.

```
abstract static void method();  // COMPILE ERROR
```

Why? Because `abstract` means *"subclass will provide the implementation"*. But static methods belong to the class, not instances. You can't override a static method in the traditional sense (you can only *hide* it). So combining `abstract` and `static` makes no sense — the "override chain" doesn't work for static methods.

Interview Question 4: Can Abstract Classes Have Private Methods?

Answer: YES — But they CANNOT be abstract

```
abstract class Animal {

    // Private method — allowed
    private void breathe() {
        System.out.println("Breathing...");
    }

    // Using private method internally
    void liveLife() {
        breathe();  // calls private method from within same class
        makeSound();
    }

    abstract void makeSound();
}
```

Why private methods are allowed:

Private methods are internal implementation details of the class. They help with code organization within the abstract class itself. The fact that the class is abstract doesn't change this.

Why private methods CANNOT be abstract:

```
private abstract void method();  // COMPILE ERROR
```

`abstract` means: *"I don't have an implementation — a subclass must provide it."* `private` means: *"Subclasses cannot see me at all."*

If the method is invisible to subclasses, they can never override it. So there's no way for the abstract method to ever get an implementation. Java refuses this combination.

Interview Question 5: Can Abstract Classes Have `final` Methods?

Answer: YES — But the final method CANNOT be abstract

```
abstract class Animal {

    // Final method — allowed, but cannot be overridden
    final void breathe() {
        System.out.println("Inhale... Exhale...");
    }

    // Abstract method — must be overridden by subclass
    abstract void makeSound();
}

class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("Woof!");
    }

    // void breathe() { ... }  // COMPILE ERROR — cannot override final method
}
```

Why `final` methods are allowed in abstract classes:

`final` means the method's implementation is fixed and cannot be changed by subclasses. This is perfectly valid in an abstract class — you're saying: *"This specific behavior is locked, but other behaviors (abstract methods) you must define yourself."*

Why `final abstract` is contradictory:

```
final abstract void method();  // COMPILE ERROR
```

`final` = cannot be overridden. `abstract` = must be overridden. Impossible to satisfy both simultaneously.

Interview Question 6: Can Abstract Classes Have NO Abstract Methods?

Answer: YES — Perfectly valid

```
abstract class Vehicle {
    String brand;

    Vehicle(String brand) {
        this.brand = brand;
    }

    void start() {
        System.out.println(brand + " is starting");
    }

    void stop() {
        System.out.println(brand + " is stopping");
    }

    // No abstract methods at all!
}

class Car extends Vehicle {
    Car(String brand) {
        super(brand);
    }
}
```

Why you'd want this:

You want to prevent direct instantiation of `Vehicle` — nobody should create a raw `Vehicle`. They should always create a specific `Car`, `Truck`, or `Motorcycle`. But all vehicles share common behavior (start, stop), so you provide full implementations for everything.

```
Vehicle v = new Vehicle("Generic");  // COMPILE ERROR
Vehicle v = new Car("Toyota");       // Works fine
```

The `abstract` keyword here acts purely as an **instantiation blocker**, not requiring any abstract methods.

---

11. POJO Classes — Complete Deep Dive

What Is a POJO?

POJO = Plain Old Java Object

A POJO is a simple Java class that:

- Uses no external framework annotations or dependencies
- Just holds data through fields
- Has constructors to initialize that data
- Has getters/setters to access that data
- May have simple utility methods

```
public class Student {

    // Fields (private for encapsulation)
    private String name;
    private int age;
    private int rollNumber;
    private String college;

    // Constructor
    public Student(String name, int age, int rollNumber, String college) {
        this.name = name;
        this.age = age;
        this.rollNumber = rollNumber;
        this.college = college;
    }

    // Default constructor
    public Student() { }

    // Getters
    public String getName() { return name; }
    public int getAge() { return age; }
    public int getRollNumber() { return rollNumber; }
    public String getCollege() { return college; }

    // Setters
    public void setName(String name) { this.name = name; }
    public void setAge(int age) { this.age = age; }
    public void setRollNumber(int rollNumber) { this.rollNumber = rollNumber; }
    public void setCollege(String college) { this.college = college; }
}
```

This is a pure POJO. It models the real-world concept of a Student.

Why POJOs Exist

Purpose: Model real-world entities in code

When you need to represent a real-world thing (Student, Order, Product, User), you need a structured container. A POJO is that container.

Database mapping:

If you have a `students` table in SQL:

```
CREATE TABLE students (
   name VARCHAR(100),
   age INT,
   roll_number INT,
   college VARCHAR(200)
);
```

You model this in Java as a POJO that mirrors the table structure. Frameworks like Hibernate/JPA can automatically map between the table and your POJO.

Data transfer:

When your backend sends data to a frontend (as JSON), the POJO is serialized:

```
Student s = new Student("Alice", 20, 101, "MIT");
// Serialized to: {"name":"Alice","age":20,"rollNumber":101,"college":"MIT"}
```

Two Architectural Flavors of POJO

Anemic Domain Model

Contains *only* data — fields, constructors, getters, setters. No business logic.

```
public class Student {
   private String name;
   private double gpa;
   private int attendancePercentage;

   // Constructor
   public Student(String name, double gpa, int attendancePercentage) {
      this.name = name;
      this.gpa = gpa;
      this.attendancePercentage = attendancePercentage;
```

```
    }

    // Only getters and setters — no logic
    public String getName() { return name; }
    public double getGpa() { return gpa; }
    public int getAttendancePercentage() { return attendancePercentage; }
    public void setName(String name) { this.name = name; }
    public void setGpa(double gpa) { this.gpa = gpa; }
    public void setAttendancePercentage(int pct) { this.attendancePercentage = pct; }
}
```

Business logic lives in a *separate* service class:

```
public class StudentService {
    public boolean isEligibleForExam(Student s) {
        return s.getAttendancePercentage() >= 75;
    }

    public String getGrade(Student s) {
        if (s.getGpa() >= 9.0) return "A+";
        if (s.getGpa() >= 8.0) return "A";
        return "B";
    }
}
```

Advantage: Clean separation. The model is pure data. Business rules are isolated and easy to test/change.

Rich Domain Model

Contains data AND business logic relevant to that entity.

```
public class Student {
    private String name;
    private double gpa;
    private int attendancePercentage;
    private List<String> enrolledCourses = new ArrayList<>();

    public Student(String name, double gpa, int attendancePercentage) {
        this.name = name;
        this.gpa = gpa;
        this.attendancePercentage = attendancePercentage;
    }
```

```
    // Getters/setters...

    // Business logic inside the model
    public boolean isEligibleForExam() {
        return this.attendancePercentage >= 75;
    }

    public String getGrade() {
        if (this.gpa >= 9.0) return "A+";
        if (this.gpa >= 8.0) return "A";
        return "B";
    }

    public void enrollInCourse(String courseName) {
        if (!enrolledCourses.contains(courseName)) {
            enrolledCourses.add(courseName);
        }
    }

    public boolean isEnrolledIn(String courseName) {
        return enrolledCourses.contains(courseName);
    }
}
```

Advantage: The object "knows" about itself. More aligned with OOP philosophy where objects have both state AND behavior.

POJO vs Other Special Java Classes

Class Type	Key Characteristic
POJO	Simple class, no framework dependency
JavaBean	POJO + serializable + no-arg constructor + follows naming convention
Entity	POJO + `@Entity` annotation (JPA/Hibernate)
DTO	Data Transfer Object — POJO used to move data between layers
Record (Java 16+)	Immutable POJO with auto-generated constructor/getters/equals/hashCode

JavaBean is a stricter form of POJO:

```
public class StudentBean implements Serializable {
    private static final long serialVersionUID = 1L;

    private String name;  // private fields

    public StudentBean() { }  // mandatory no-arg constructor

    // getters/setters following naming convention: getName, setName
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}
```

Java Record (modern immutable POJO):

```
record Student(String name, int age, int rollNumber, String college) { }
// Auto-generates: constructor, getters, equals(), hashCode(), toString()

Student s = new Student("Alice", 20, 101, "MIT");
s.name();     // getter (no "get" prefix in records)
s.age();
```

Quick Reference Summary

ONE PUBLIC CLASS PER FILE

• JVM needs unambiguous entry point
• Filename = public class name = predictable loading

WRAPPER CLASSES

• Every primitive has a class: int→Integer, double→Double, etc.
• Needed for Collections, null support, utility methods
• Object: heap allocation + reference overhead → slower than primitive

AUTOBOXING

• primitive → Object (compiler inserts Integer.valueOf())
• Triggers on: assignment, method calls, arithmetic

UNBOXING

• Object → primitive (compiler inserts .intValue())
• Watch out: null object + unboxing = NullPointerException!

== vs equals()

• == compares references (addresses) for objects
• == compares values directly for primitives
• Always use .equals() to compare object contents

INTEGER CACHING

• -128 to 127: same object reused (valueOf returns cached)
• Outside range: new object created each time
• Never rely on == for Integer comparison

ABSTRACT CLASSES

• Cannot instantiate directly (abstract keyword)
• CAN have: constructors, static members, private methods, final methods
• CANNOT be: final (contradicts abstract)
• Abstract methods CANNOT be: private or final (contradicts override requirement)
• Can have zero abstract methods (pure instantiation blocker)

POJO

• Plain Old Java Object — simple data container
• No framework dependency
• Has: fields, constructors, getters/setters
• Anemic model: data only | Rich model: data + business logic